

Open in app ↗

Sign up

Sign in

Medium

🔍 Search

DevOps.dev · [Follow publication](#)

★ Member-only story

The Missing Layer: Architecting NVIDIA GPUs for Kubernetes

13 min read · May 21, 2026



L Singh

Follow



Listen



Share



Your GPU is there. Your pod is Pending . Your bill is ticking.

You spin up a GPU node pool on EKS or AKS, the nodes come up, the hardware is exactly what you paid for. You deploy your first training pod. It will not schedule.

`kubectl describe` shows the line you were hoping not to see: `Insufficient`

`nvidia.com/gpu` . You quietly open a new browser tab. If you have been there, you know exactly what comes next. If you have not, you will be soon enough.

A GPU node pool gives Kubernetes worker nodes with GPU hardware, but it **does not automatically mean Kubernetes can allocate the GPU to a container**. There is a full software stack sitting between the physical GPU and the pod. For NVIDIA GPUs, that stack usually includes the NVIDIA driver, NVIDIA Container Toolkit, NVIDIA Kubernetes device plugin, GPU Feature Discovery, DCGM Exporter, and sometimes MIG Manager.

Kubernetes does have stable support for scheduling GPUs through device plugins. A pod consumes a GPU by requesting an extended resource such as `nvidia.com/gpu` in its resource limits. But that resource only shows up in Kubernetes after the NVIDIA device plugin advertises it on the node.

Where this shows up

If you are working on model training, fine-tuning, batch ML pipelines, real-time inference services, internal AI platforms, or the Neo Clouds offering GPU capacity as a product, the same stack sits underneath all of it. The same vocabulary keeps coming up whether you are analysing requirements, defining a cluster design, implementing the stack, or operating it at scale. Driver, toolkit, device plugin, operator, MIG, DCGM, `nvidia.com/gpu` . Get fluent in those and the rest of an AI infrastructure project gets a lot easier.

The mental model

The easiest way to think about GPU scheduling in Kubernetes is to separate **hardware capacity** from **Kubernetes resource visibility**. These are two parallel tracks, and confusing them is the source of most early problems.



Create an account to read the full story.

The author made this story available to Medium members only.
If you're new to Medium, create a new account to read this story on us.

Continue in app

Or, continue in mobile web



Sign up with Google



Sign up with Facebook



Sign up with email

Already have an account? [Sign in](#)



Follow

Published in DevOps.dev

25K followers · Last published 2 days ago

DevOps.dev is a community of DevOps enthusiasts sharing insight, stories, and the latest development in the field.



Follow

Written by L Singh

11 followers · 6 following

